RISC - V Verification Project

Index / Table of Contents :

1 Project Notes — CPU Memory & PC Mechanism

1 Program Counter (PC)

- Definition: A CPU register that holds the address of the next instruction to execute.

- Function:
    1. Points to the current instruction address.
    2. Sends address to Instruction Memory / I-Cache.
    3. Updates each cycle:
        - Sequential instruction: PC + 4 (RV32I).
        - Branch / Jump: PC updated to target address.
- Location: Inside CPU core (register).
- DV Checks:
    - Verify PC increments correctly.
    - Check PC updates correctly for branches/jumps.
- Analogy: PC = finger pointing to the next recipe card in a cookbook.

---

2 Instruction Memory (I-Mem)

- Definition: Memory that stores program instructions (machine code).
- Function:
    1. Receives address from PC.
    2. Returns instruction to CPU for decoding and execution.
- Location in real CPU:
    - Fast: Inside CPU as L1 Instruction Cache.
    - Large & slow: Main memory / DRAM.
- DV Checks:
    - Verify fetched instruction matches golden model.
- Analogy: I-Mem = library of recipe cards. CPU asks for the next recipe.

---

3 Data Memory (D-Mem)

- Definition: Memory that stores program data (variables, arrays, results).
- Function:
    - Load: CPU reads data from address.
    - Store: CPU writes data to address.
- Control Signals:
    - dmem_addr → address to read/write.
    - dmem_we → write enable.

- dmem_wdata → data to write.

- Location in real CPU:
  - Fast: Inside CPU as L1 Data Cache.
  - Large & slow: Main memory / DRAM.
- DV Checks:
  - Verify load/store data correctness.
  - Verify correct memory addresses are accessed.
- Analogy: D-Mem = pantry storing ingredients (data) used in recipes.

4️⃣ Summary Table

Component	Role	Mechanism	Location	DV Relevance
PC	Holds next instruction address	Sends address to I-Mem; updates sequentially or by branch/jump	Inside CPU	Verify correct instruction fetch & branch/jump handling
Instruction Memory (I-Mem)	Stores instructions	Receives PC → returns instruction	L1 I-Cache or Main Memory	Verify fetched instruction matches expected
Data Memory (D-Mem)	Stores data	Receives address + control → read/write	L1 D-Cache or Main Memory	Verify load/store addresses & data correctness

---

✅ **Takeaway:**

- PC drives instruction fetch.
- I-Mem provides instructions.
- D-Mem provides/stores data.
- Verification focuses on architecturally visible behavior, not internal hardware implementation.

1️⃣ Program Counter Output (pc_out) → control flow correctness

What is pc_out?

logic [31:0] pc_out;

pc_out is a **verification-visible copy** of the CPU's internal Program Counter (PC).

Internally, the CPU maintains a private pc register.
For DV purposes, this value is **exported** using pc_out.

What does the PC represent?

- Holds the **address of the current instruction**
- Controls **instruction fetch**
- Determines **program flow**

Normally:

PC_next = PC + 4

Except for:

- Branches
- Jumps
- Exceptions

Why expose pc_out for DV?

In real silicon:

- PC is **not visible externally**

In verification:

- PC visibility is **critical**

DV uses pc_out to verify:

- Correct instruction sequencing
- Correct branch/jump targets
- PC alignment (4-byte aligned)
- Off-by-4 bugs
- Pipeline flush correctness

DV Checks Using pc_out

✓ PC increments by 4 for sequential instructions
✓ PC updates correctly on branch taken
✓ PC updates correctly on JAL / JALR
✓ PC is always 4-byte aligned

Example assertion:

assert ((pc_out & 32'h3) == 0);

2️⃣ **Register File Output (regs_out[0:31]) regs_out[] → architectural register correctness**

What is regs_out?

logic [31:0] regs_out [0:31];

regs_out exposes **all 32 architectural registers** of the RISC-V CPU:

Index	Register
0	x0 (hardwired zero)
1–31	General-purpose registers

Each register is **32 bits wide** (RV32).

Why expose registers for DV?

In real CPUs:

- Register file is internal and hidden

In DV:

- Register visibility is essential to verify:
    - ALU results
    - Immediate handling
    - Load data correctness
    - Writeback logic
    - Destination register selection

Example

Instruction:

ADD x3, x1, x2

DV check:

regs_out[3] == regs_out[1] + regs_out[2]

If mismatch:

- Decode bug
- ALU bug
- Writeback bug

Special Rule: x0 Register

RISC-V ISA rule:

x0 must always be 0

DV assertion:

assert (regs_out[0] == 0);

This catches:

- Illegal writes to x0
- Incorrect writeback gating

→So DV rule #1:

Only verify memory when the instruction actually accesses memory.

Where is “memory” in your verification?

You do NOT directly peek inside DUT memory.

Instead, you observe:

dmem_addr

dmem_wdata

dmem_wstrb

dmem_we

dmem_rdata

This is the memory interface, not the memory itself.

Think of it like this:

CPU ---> Memory Interface ---> Memory

DV checks what the CPU sends and receives.

→DV principle:

Always check architectural state, not internal wires.

→Very Important Insight (Interview Gold ⭐)

Memory is checked indirectly via loads and stores, not continuously.

- Stores → update expected memory
- Loads → verify memory via registers

Memory checking is done in:

- ✓ Scoreboard
- ✓ Reference model
- ✓ Monitor

Testbench Clocking Block & Signal Directions

Purpose:
Clocking blocks in SystemVerilog help synchronize signal sampling and driving with the DUT clock,

providing a structured interface for verification. They are mainly used in DV to simplify **signal access, timing control, and assertions**.

1 Signal Ownership vs Clocking Block Direction

Concept	Who Generates the Signal in Real CPU	Clocking Block Direction (TB Perspective)
`imem_addr`	CPU generates PC-based instruction address	`input` → TB reads this signal
`imem_rdata`	Memory provides instruction data	`output` → TB drives this signal
`dmem_addr`	CPU generates data memory address	`input` → TB reads
`dmem_wdata`	CPU generates store data	`input` → TB reads
`dmem_rdata`	Memory provides load data	`output` → TB drives
`pc_out`	CPU internal PC value	`input` → TB reads
`regs_out[0:31]`	CPU register file values	`input` → TB reads

Key Idea:

Clocking block directions are **from the testbench perspective**, not from hardware perspective. They only define **who the TB can read or drive**, not the real owner in the CPU.

2 Why TB Drives Some Signals

- TB often models **memory or external devices**.
- Example: CPU asks “Give me instruction at address X”.
    - CPU drives imem_addr.
    - TB responds with instruction: drives imem_rdata.
- TB drives **load responses** from memory (dmem_rdata) or device outputs.
- CPU drives all addresses and write-data signals.

---

3 Correct Mapping for RISC-V Verification

CPU → Drives:

- imem_addr (Instruction fetch address)
- dmem_addr (Memory access address)
- dmem_wdata (Store data)
- dmem_wstrb (Store mask)
- dmem_we (Write enable)

TB → Drives:

- imem_rdata (Instruction data)

- dmem_rdata (Load data)

TB → Reads:

- All CPU-driven signals for monitoring and coverage

4️⃣ Common DV Checks Using Clocking Block

- **x0 register never changes:** assert regs_out[0] == 0
- **PC alignment check:** assert (pc_out & 32'h3) == 0
- **Memory write correctness:** if dmem_we, dmem_wstrb != 0
- **Instruction opcode coverage:** coverpoint on imem_rdata[6:0]
- **Load/store memory correctness:** monitor dmem_addr and dmem_rdata

---

5️⃣ Key Takeaways for DV

- Clocking block **simplifies reading and driving signals** in a structured, cycle-accurate manner.
- **Signal directions are TB perspective only**; always know **actual hardware ownership**.
- TB often acts as a **memory model**: it responds with data when CPU drives an address.
- This is essential for **writing assertions, coverage, and scoreboards** in CPU verification.

6️⃣ Interview Note

“In DV, understanding signal ownership versus TB driving direction is critical. CPU drives addresses and data for writes, TB drives responses for reads, while clocking block input/output only defines TB access, not hardware ownership.”

Assertions (SVA) — CPU Verification Notes

Purpose:
Assertions are automatic checks embedded in the testbench or interface that **verify critical CPU rules** at every clock cycle. They help catch bugs early by monitoring architectural state, memory, and control signals.

1️⃣ x0 Must Always Be Zero

- **Signal:** regs_out[0]
- **Rule:** RISC-V register x0 is **always zero**. Cannot be written by any instruction.
- **Check:** Assertion triggers an error if x0 changes.
- **DV Importance:** Detects illegal writes to x0 register.
- **SystemVerilog Example:**

```
assert property (@(posedge clk) disable iff(reset) regs_out[0] == 0)
```

else $error("x0 register changed!");

---

2 PC Must Be 4-Byte Aligned

- **Signal:** pc_out
- **Rule:** Instructions are 32-bit → PC must be **multiple of 4**.
- **Check:** Lower 2 bits of PC (pc_out[1:0]) should be zero.
- **DV Importance:** Ensures correct instruction fetch address.
- **Example:** If PC = 0x1003 → misaligned → assertion triggers.

---

3 No Memory Writes During Reset

- **Signal:** dmem_we
- **Rule:** CPU should not write to memory during reset.
- **Check:** Whenever reset is high, dmem_we must be 0.
- **DV Importance:** Prevents memory corruption during reset.

---

4 Store Must Have Valid Write Strobe

- **Signals:** dmem_we, dmem_wstrb
- **Rule:** On a store, at least one byte must be enabled for writing.
- **Check:** If dmem_we is active, dmem_wstrb ≠ 0.
- **DV Importance:** Ensures store instructions actually write data.

Summary Table

Assertion Name	Checks	DV Importance
x0_never_changes	x0 register always 0	Detect illegal writes
pc_alignment	PC multiple of 4	Correct instruction fetch
no_write_during_reset	Memory write disabled during reset	Preserve memory integrity
store_has_strobe	Store write enables ≥1 byte	Prevent store errors

Analogy

Think of it like a teacher grading a student:

Role	Analogy
DUT	Student performing calculations (CPU executes instructions)
TB	Teacher giving problems and checking answers (testbench generates expected results)
Scoreboard	Teacher's answer sheet (golden model)

Transaction Items in RISC-V CPU Verification (DV Notes)

1. Why Transaction Items Exist (Big Picture)

In functional verification, transaction items are *testbench-level data structures* used to:

- Represent instructions and memory operations
- Carry expected / observed information
- Help the scoreboard predict and compare DUT behavior
- Improve debug, coverage, and clarity

⚠️ Transaction items do NOT drive hardware signals directly. They are verification abstractions, not physical CPU components.

---

2. Key Rule (Very Important)

Even if a value is generated by the CPU (DUT), the testbench may still store it inside a transaction for checking and tracking purposes.

This is the root of most confusion for beginners.

"Fields like instr and pc are declared rand because the same transaction class is reused across multiple sequences. Even if a particular sequence assigns fixed values or only observes them, other sequences rely on constrained randomization for coverage and corner-case testing."

---

3. riscv_txn — Instruction-Level Transaction

```
class riscv_txn extends uvm_sequence_item;
   rand bit [31:0] instr;
   rand bit [31:0] pc;

   bit [4:0]  rd;
   bit [4:0]  rs1;
```

```
  bit [4:0]  rs2;
  bit [31:0] op1;
  bit [31:0] op2;
  bit [31:0] alu_res;
  bit        branch_taken;
endclass
```

3.1 instr — Instruction Stimulus (Randomized)

```
rand bit [31:0] instr;
```

- Generated by TB
- Consumed by DUT
- Drives instruction memory (imem_rdata)
- Used to explore different instruction types (ADD, LW, BEQ, etc.)

✅ This is true stimulus

3.2 pc — Instruction Context (NOT Driving DUT)

```
rand bit [31:0] pc;
```

🔴 Common misunderstanding

“PC is generated by DUT, why randomize it?”

✅ Correct understanding

- pc here is NOT driving the DUT
- It represents the expected or tagged PC for this instruction
- Used by:
  - Scoreboard
  - Logging
  - Correlation of instruction ↔ execution

📌 The DUT still generates pc_out internally

📌 TB stores pc only to check correctness

Think of pc as a label, not a control signal

3.3 Decoded Fields (rs1, rs2, rd)

```
bit [4:0] rs1, rs2, rd;
```

- NOT randomized
- Filled by:
  - Monitor
  - Reference model
  - Scoreboard

Used for:

- Debugging
- Coverage
- Clear failure messages

Example:
Expected: rs1 = x3
Actual: rs1 = x4 ❌

3.4 Operand and Result Fields
bit [31:0] op1, op2, alu_res;
bit   branch_taken;
Purpose:

- Store expected or observed values
- Help scoreboard compare:
  - ALU results
  - Branch decisions

They allow statements like:
ALU mismatch for ADD at PC 0x104
Expected: 0x20
Actual: 0x1C
✅ Improves debug quality
✅ Not hardware drivers

---

4. mem_txn — Memory-Level Transaction

```
class mem_txn extends uvm_sequence_item;
   rand bit [31:0] addr;
   rand bit [31:0] wdata;
   rand bit [3:0]  wstrb;
   rand bit        is_write;

   bit [31:0] rdata;
endclass
```

4.1 Why Memory Transactions Exist
Memory behavior must be verified for:

- Loads
- Stores
- Byte/half/word accesses
- Sign/zero extension
- Correct addressing

4.2 addr, wdata, is_write — Observed or Expected

🔴 Another common confusion:

“CPU generates address — why randomize it?”

✅ Reality:

- These fields represent:
  - Expected memory access
  - OR observed memory access
- Filled by:
  - Memory driver (for stimulus)
  - Monitor (for observation)

📌 They do not override DUT behavior

4.3 rdata — Observed Memory Read

bit [31:0] rdata;

- Captured from DUT
- Compared against golden memory model

Example:

Load from 0x200

Expected: 0x55

Actual: 0x55 ✓

5. Who Generates What? (Clear Separation)

Item	Generated by
`instr`	Testbench
`pc` (txn)	TB reference / tracking
`pc_out`	DUT
`addr` (mem_txn)	DUT (observed)
`rdata`	Memory model
`regs_out`	DUT

6. How Prediction Works (Core DV Concept)

1. TB generates instruction
2. Reference model executes instruction in software
3. Expected:
   - PC
   - Registers

    - Memory updates
4. DUT executes instruction
5. Monitor captures actual behavior
6. Scoreboard compares:

Expected == Actual ✓

⚠️ This is why DV engineers must understand the design

7. Why This Design Is Industry-Standard

✓ Scales to pipelines & OoO cores

✓ Enables clear debug

✓ Supports coverage-driven verification

✓ Separates stimulus, observation, and checking

---

8. Final Golden Rule (Put This in Resume)

“Transaction items do not control hardware. They carry expected and observed architectural information so the scoreboard can validate DUT behavior against a golden reference model.”

--→

In real hardware, instructions are stored in memory at fixed addresses and fetched by the CPU using the program counter. In the verification environment, physical instruction memory does not exist. Instead, the testbench models instruction memory behavior by associating instruction values with expected PC addresses. The CPU independently generates the PC, and the testbench responds with the corresponding instruction only when the requested address matches the expected PC. Any mismatch indicates a functional error in control flow and is flagged during verification.

1 Project Overview

This project verifies a RISC-V CPU core using SystemVerilog and UVM methodology. The verification environment validates instruction execution, register behavior, memory access, and control flow of the processor.

The following aspects are verified:

- Instruction fetch and decode
- Program Counter (PC) update logic
- Register file read/write correctness
- ALU operations (ADD, ADDI)
- Load/Store memory operations
- Branch and Jump instructions
- Architectural state consistency
- Reset behavior
- Directed + stress test scenarios
- Functional coverage for instruction types

This verification follows industry-standard CPU verification methodology, similar to real SoC processor validation environments.

2 RISC-V ISA & CPU Concept

RISC-V is an **open-standard Reduced Instruction Set Computer (RISC) architecture**.

i. CPU Execution Model

Each instruction goes through:

1. **Fetch** – instruction read from instruction memory
2. **Decode** – opcode, registers, immediate extraction
3. **Execute** – ALU / branch decision
4. **Memory** – load/store access
5. **Writeback** – register update

ii. Instruction Types Verified

Instruction Type	Example	Function
R-type	ADD	Register-to-register ALU
I-type	ADDI	Immediate ALU
Load	LW	Load from memory
Store	SW	Store to memory
Branch	BEQ	Conditional branch
Jump	JAL	Unconditional jump

iii. Program Counter (PC) Behavior

- Normally increments by +4
- Modified by branch/jump instructions
- Verified cycle-by-cycle in scoreboard

3 RISC-V Interface Signals

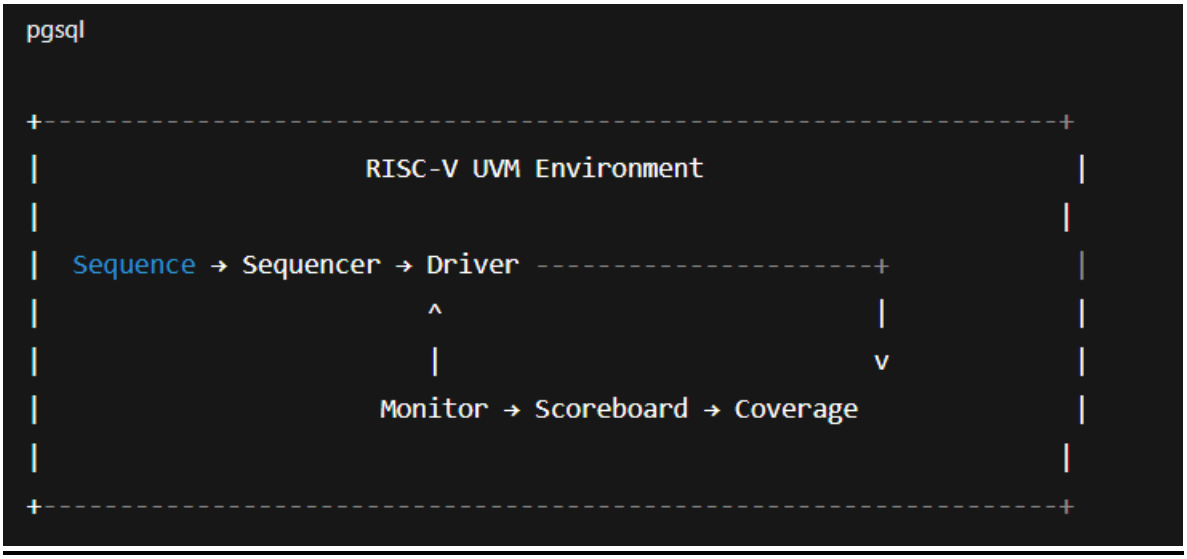
Signal	Direction	Description
clk	Input	System clock
reset	Input	Synchronous reset
pc_out	Output	Current Program Counter
imem_rdata	Input	Instruction fetched
dmem_addr	Output	Data memory address
dmem_wdata	Output	Store data
dmem_rdata	Input	Load data
dmem_we	Output	Write enable
dmem_wstrb	Output	Byte enable
regfile	Internal	Register file state

4️ Features Verified

Instruction fetch correctness :

- Opcode decoding
- Register read/write correctness
- x0 register remains hard-wired to zero
- ALU result accuracy
- Immediate sign-extension logic
- Load/Store address calculation
- Memory data correctness
- Branch decision logic
- Jump target calculation
- PC update sequencing
- Reset behavior
- Back-to-back instruction execution
- RAW dependency scenarios
- No X-propagation in architectural state

5 UVM Verification Architecture



Components:

- Sequence Item – instruction, PC
- Sequences – reset, rtype, itype, load, store, branch, jump
- Driver – drives instructions into DUT
- Monitor – observes instruction & memory activity
- Scoreboard – reference CPU model
- Coverage – instruction & execution coverage
- Agent – driver + monitor
- Environment – agent + scoreboard + coverage
- Tests – directed + stress tests

6 Verification Flow

Phase	Test Type	Purpose
Smoke	Reset + NOP	Connectivity & sanity
Directed	Instruction tests	Functional correctness
Load/Store	Memory tests	Address & data integrity
Control Flow	Branch/Jump	PC update correctness
Stress	Back-to-back ops	Pipeline & dependency
Coverage	Functional	Scenario completeness
Regression	Multi-seed	Stability & sign-off

7️⃣ Testbench Components

7.1 Sequence Item (riscv_txn)

Contains:

Instruction word

Program Counter

Decoded fields (rs1, rs2, rd – optional)

7.2 Driver

Drives instructions to DUT

Synchronizes with clock

Handles reset sequencing

Ensures correct instruction timing

7.3 Monitor

Samples:

PC value

Instruction word

Memory read/write signals

Publishes:

Instruction transactions

Memory transactions

7.4 Scoreboard

Implements reference RISC-V architectural model:

exp_regs[rd] = exp_regs[rs1] + exp_regs[rs2];

exp_pc += 4;

Checks:

- PC correctness
- Register write correctness
- Memory load/store correctness
- Architectural state consistency

7.5 Coverage

Coverpoints:

- Instruction opcode
- Instruction type
- Register usage
- Load vs Store
- Branch taken vs not taken

Cross Coverage:

- opcode × instruction type
- register × operation

7.6 Assertions

Assertions ensure:

PC alignment

No invalid memory access

Correct reset behavior

8. Final Verdict

Directed Tests + Stress Tests + Scoreboard + Coverage

= Complete RISC-V CPU Verification Sign-off

Scoreboard ensures functional correctness.

Coverage ensures scenario completeness.

Tests ensure real-world execution confidence. This is industry-standard CPU verification flow.

9. Conclusion

This RISC-V verification project validates complete CPU functionality using:

- UVM-based stimulus
- Reference-model scoreboard
- Functional coverage
- Directed + stress tests

It demonstrates strong understanding of CPU architecture, UVM methodology, and industry-level verification practices.

RISC – V Concepts

1️⃣ CPU Memory & PC Mechanism

A CPU executes a program by repeatedly:

1. Fetching an instruction
2. Executing it
3. Updating architectural state

Three core concepts enable this:

Program Counter (PC)

Instruction Memory (I-Mem)

Data Memory (D-Mem)

👉 Verification checks **what the CPU exposes architecturally**, not how it is implemented internally.

2️⃣ Program Counter (PC)

The Program Counter is a CPU register that holds the address of the next instruction to execute.

Functions of PC

1. Points to the current instruction address
2. Sends address to Instruction Memory
3. Updates every instruction

PC Update Rules (RV32I)

- Normal instruction → PC = PC + 4
- Branch taken → PC = branch_target
- Jump → PC = jump_target

Location

- Inside CPU core (register)
- Not visible externally in real silicon

DV View of PC :

Why PC Matters in Verification

PC controls:

Instruction order

Control flow correctness

Branch & jump behavior

DV Checks :

PC increments by 4

Correct branch target

Correct jump target

No off-by-4 errors

Analogy :

PC = finger pointing to the next recipe card in a cookbook

3️⃣ Instruction Memory (I-Mem)

What is Instruction Memory? Memory that stores machine instructions.

Function

1. Receives address from PC
2. Returns instruction to CPU

Real CPU Location

L1 Instruction Cache (fast)

DRAM (slow)

DV Perspective of I-Mem : There is **NO real instruction memory** in the testbench.

How DV Models I-Mem

CPU drives instruction address

Testbench responds with instruction data

DV Check : ✓ Instruction returned matches expected instruction for that PC

Analogy : I-Mem = library of recipe cards

4️⃣ Data Memory (D-Mem)

What is Data Memory? Memory that stores **program data** (variables, arrays, results).

Operations

- Load → read data
- Store → write data

Control Signals

- dmem_addr → address
- dmem_we → write enable
- dmem_wdata → write data
- dmem_rdata → read data

Real CPU Location

- L1 Data Cache
- DRAM

DV Perspective of D-Mem

Key Rule: DV does NOT peek inside memory arrays.

DV only observes the memory interface: CPU → dmem_* → Memory

DV Checks

Correct address accessed

Correct data written

Correct data loaded

Analogy : D-Mem = pantry storing ingredients used in recipes

Takeaway (PC + I-Mem + D-Mem)

- PC drives instruction fetch
- I-Mem provides instructions
- D-Mem stores program data
- DV checks architectural behavior, not internal RAMs

5 Program Counter Output (pc_out)

What is pc_out? logic [31:0] pc_out;

A verification-visible copy of the internal PC

Exported only for DV

Why Expose pc_out?

PC is invisible in real silicon, but DV needs it to:

- Track instruction sequence
- Validate control flow
- Debug branch/jump bugs

DV Checks Using pc_out

- PC increments by 4
- PC updates correctly on branch/jump
- PC always 4-byte aligned

Example assertion: assert ((pc_out & 32'h3) == 0);

---

6 Register File Output (regs_out[0:31])

What is regs_out?

logic [31:0] regs_out [0:31];

Exposes all **32 RISC-V architectural registers**:

x0 → hardwired zero

x1–x31 → general purpose

Why Expose Registers for DV?

To verify:

ALU results

Immediate handling

Load writeback

Destination register selection

Example

Instruction: ADD x3, x1, x2

DV check: regs_out[3] == regs_out[1] + regs_out[2]

Special Rule: x0 Register

RISC-V rule: x0 must always be zero

DV assertion: assert (regs_out[0] == 0);

Catches : Illegal writes, Writeback bugs

7️ Memory Verification Rule (Interview Gold ⭐)

DV Rule #1 : Only verify memory when an instruction accesses memory.

Memory is checked:

- On stores → update expected memory
- On loads → compare expected vs actual

Memory is verified indirectly, not continuously

8️ Clocking Block & Signal Direction

Clocking blocks Purpose :

Synchronize TB & DUT

Control sampling & driving timing

Improve assertion reliability

Important: Clocking directions are from testbench perspective only.

Signal Ownership

CPU Drives:

imem_addr, dmem_addr, dmem_wdata, dmem_wstrb, dmem_we

Testbench Drives:

imem_rdata, dmem_rdata

TB reads everything else.

DV Checks Using Clocking Block

- x0 never changes
- PC alignment
- Valid store strobes
- Load/store correctness
- Opcode coverage

---

9 Assertions (SVA)

1 x0 Must Always Be Zero

```
assert property (@(posedge clk) disable iff(reset)
                 regs_out[0] == 0);
```

2 PC Must Be Aligned

```
assert ((pc_out & 32'h3) == 0);
```

3 No Memory Writes During Reset

```
assert property (@(posedge clk) reset |-> !dmem_we);
```

4 Valid Store Strobe

```
assert property (@(posedge clk)
                 dmem_we |-> (dmem_wstrb != 0));
```

---

10 Transaction Items

Why Transaction Items Exist

They:

- Represent instructions and memory ops
- Carry expected & observed values
- Help scoreboard compare behavior

■ They DO NOT drive hardware

i. riscv_txn — Instruction Transaction

rand bit [31:0] instr;

rand bit [31:0] pc;

bit [4:0] rs1, rs2, rd;

bit [31:0] op1, op2, alu_res;

bit branch_taken;

Key Insight :

• instr → stimulus

• pc → label/context (NOT driving DUT)

Think: pc here = tag, not control

ii. mem_txn — Memory Transaction

rand bit [31:0] addr;

rand bit [31:0] wdata;

rand bit [3:0] wstrb;

rand bit is_write;

bit [31:0] rdata;

Used to:

- Track loads/stores
- Compare memory behavior

1️⃣1️⃣ Prediction Flow (Golden DV Concept)

1. TB generates instruction
2. Reference model executes instruction
3. Expected PC / regs / memory updated
4. DUT executes instruction
5. Monitor captures actual behavior
6. Scoreboard compares expected vs actual

Match → PASS

Mismatch → BUG

1️⃣2️⃣ Final Golden Rule

“Transaction items do not control hardware. They carry expected and observed architectural information so the scoreboard can validate DUT behavior against a golden reference model.”

⭐ Final Insights

In real hardware:

- Instructions live in memory
- CPU fetches using PC

In verification:

- No real instruction memory exists
- TB models instruction memory behavior
- CPU generates PC
- TB responds with instruction for that PC
- Any mismatch = control flow bug